

LAPORAN PRAKTIKUM
STRUKTUR DATA
BINARY TREE & GRAPH TRAVERSAL

disusun Oleh:

RIFKI MAULANA
NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T
ASISTEN PRAKTIKUM : ZAHRAN AZARIA ANVAYA



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG, 11 JUNI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Binary Tree dan Graph Traversal**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Binary Tree dan Graph Traversal.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 11 Juni 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	Error! Bookmark not defined.
2.2 Kode Program.....	Error! Bookmark not defined.
2.2.1 NodeDLL	Error! Bookmark not defined.
2.2.2 InsertDLL.....	Error! Bookmark not defined.
2.2.3 PenelusuranDLL	Error! Bookmark not defined.
2.2.4 HapusDLL	Error! Bookmark not defined.
BAB III KESIMPULAN	8
3.1 Kesimpulan.....	8
DAFTAR PUSTAKA.....	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data adalah cara untuk menyimpan dan mengatur data sehingga Anda dapat menggunakannya secara efisien. Tree adalah struktur data berbentuk hierarki yang terdiri dari kumpulan node (simpul) yang saling terhubung. Binary Tree adalah tree di mana setiap node memiliki maksimal dua anak, yaitu left child dan right child. Graph adalah struktur data yang terdiri dari kumpulan vertex (simpul) dan edge (sisi) yang menghubungkan simpul-simpul tersebut. Untuk mengakses seluruh node, digunakan teknik traversal. Traversal yang ada pada Tree yaitu Preorder, Inorder, Postorder. Traversal yang ada pada Graph yaitu Depth First Search (DFS), Breadth First Search (BFS)

1.2 Tujuan Praktikum

- Memahami konsep Binary Tree pada pemrograman Java
- Memahami konsep Traversal Graph pada pemrograman java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penggunaan dan pemrograman Binary Tree.
- Menambah pemahaman mengenai penggunaan dan pemrograman Graph Traversal.

BAB II PEMBAHASAN

2.1 Kode Program Node

```
package pekan9_2511533007;

public class Node_2511533007 {
    int data_3007;
    Node_2511533007 left_3007;
    Node_2511533007 right_3007;
    public Node_2511533007(int data_3007) {
        this.data_3007 = data_3007;
        left_3007 = null;
        right_3007 = null;
    }
    public void setLeft_3007(Node_2511533007 node_3007) {
        if (left_3007 == null)
            left_3007 = node_3007;
    }
    public void setRight_3007(Node_2511533007 node_3007) {
        if (right_3007 == null)
            right_3007 = node_3007;
    }
    public Node_2511533007 getLeft_3007() {
        return left_3007;
    }
    public Node_2511533007 getRight_3007() {
        return right_3007;
    }
    public int getData_3007() {
        return data_3007;
    }
    public void setData_3007(int data_3007) {
        this.data_3007 = data_3007;
    }
    void printPreorder_3007(Node_2511533007 node_3007) {
        if (node_3007 == null)
            return;
        System.out.print(node_3007.data_3007 + " ");
        printPreorder_3007(node_3007.left_3007);
        printPreorder_3007(node_3007.right_3007);
    }
    void printPostorder_3007(Node_2511533007 node_3007) {
        if (node_3007 == null)
            return;
        printPostorder_3007(node_3007.left_3007);
        printPostorder_3007(node_3007.right_3007);
        System.out.print(node_3007.data_3007 + " ");
    }
    void printInorder_3007(Node_2511533007 node_3007) {
        if (node_3007 == null)
            return;
        printInorder_3007(node_3007.left_3007);
        System.out.print(node_3007.data_3007 + " ");
        printInorder_3007(node_3007.right_3007);
    }
    public String print_3007() {
        return this.print_3007("", true, "");
    }
    public String print_3007(String prefix_3007, boolean isTail_3007, String sb_3007) {
        if (right_3007 != null) {
            right_3007.print_3007(prefix_3007 + (isTail_3007 ? "| " : " "), false,
sb_3007);
        }
        System.out.println(prefix_3007 + (isTail_3007 ? "\\-- " : "/-- ") + data_3007);
        if (left_3007 != null) {
            left_3007.print_3007(prefix_3007 + (isTail_3007 ? " " : "| "), true, sb_3007);
        }
        return sb_3007;
    }
}
```

Kode Program 2.1

Method `Node_2511533007` merupakan constructor untuk membuat simpul baru. Method `setLeft_3007` dan `setRight_3007` berfungsi mengisi anak kiri dan kanan jika masih kosong, sedangkan `getLeft_3007`, `getRight_3007`, dan `getData_3007` adalah *getter* untuk mengambil simpul anak serta data angka di dalamnya. Method `setData_3007` digunakan untuk mengubah nilai data simpul. Untuk penelusuran, `printPreorder_3007` mencetak data dengan urutan parent, kiri, kemudian kanan, `printPostorder_3007` dengan urutan kiri, kanan, kemudian parent, dan `printInorder_3007` dengan urutan kiri, parent, kemudian kanan. Terakhir, method `print_3007()` tanpa parameter bertindak sebagai awal untuk memanggil method rekursif `print_3007` berparameter untuk mencetak visualisasi struktur binary tree.

2.2 Kode Program BTree

```
package pekan9_2511533007;

public class BTree_2511533007 {
    private Node_2511533007 root_3007;
    private Node_2511533007 currentNode_3007;
    public BTree_2511533007() {
        root_3007 = null;
    }
    public boolean search_3007(int data_3007) {
        return search_3007(root_3007, data_3007);
    }
    private boolean search_3007(Node_2511533007 node_3007, int data_3007) {
        if (node_3007.getData_3007() == data_3007)
            return true;
        if (node_3007.getLeft_3007() != null)
            if (search_3007(node_3007.getLeft_3007(), data_3007))
                return true;
        if (node_3007.getRight_3007() != null)
            if (search_3007(node_3007.getRight_3007(), data_3007))
                return true;
        return false;
    }
    public void printInorder_3007() {
        root_3007.printInorder_3007(root_3007);
    }
    public void printPreOrder_3007() {
        root_3007.printPreorder_3007(root_3007);
    }
    public void printPostOrder_3007() {
        root_3007.printPostorder_3007(root_3007);
    }
    public Node_2511533007 getRoot_3007() {
        return root_3007;
    }
    public boolean isEmpty_3007() {
        return root_3007 == null;
    }
    public int countNodes_3007() {
        return countNodes_3007(root_3007);
    }
    private int countNodes_3007(Node_2511533007 node_3007) {
        int count_3007 = 1;
        if (node_3007 == null) {
            return 0;
        } else {
            count_3007 += countNodes_3007(node_3007.getLeft_3007());
            count_3007 += countNodes_3007(node_3007.getRight_3007());
        }
    }
}
```

```

        return count_3007;
    }
}

public void print_3007() {
    root_3007.print_3007();
}

public Node_2511533007 getCurrent_3007() {
    return currentNode_3007;
}

public void setCurrent_3007(Node_2511533007 node_3007) {
    this.currentNode_3007 = node_3007;
}

public void setRoot_3007(Node_2511533007 root_3007) {
    this.root_3007 = root_3007;
}
}

```

Kode Program 2.2

Method BTree_2511533007 adalah constructor untuk menginisialisasi tree kosong. Method search_3007 publik berfungsi sebagai pemanggil awal untuk method search_3007 privat yang mencari keberadaan suatu data di dalam tree secara rekursif. Method printInorder_3007, printPreOrder_3007, dan printPostOrder_3007 berfungsi memicu pencetakan data seluruh simpul sesuai jenis traversalnya masing masing dimulai dari akar. Method getRoot_3007 mengambil simpul akar, isEmpty_3007 memeriksa status kosongnya tree, dan print_3007 mencetak bentuk visual tree. Method countNodes_3007 publik memicu method countNodes_3007 privat untuk menghitung total simpul secara rekursif. Sementara itu, getCurrent_3007, setCurrent_3007, dan setRoot_3007 bertugas mengambil, mengubah simpul posisi saat ini, serta menetapkan simpul akar tree.

2.3 Kode Program BTreeDriver

```

package pekan9_2511533007;

public class BtreeDriver_2511533007 {
    public static void main(String[] args) {
        //Membuat Pohon
        BTree_2511533007 tree_3007 = new BTree_2511533007();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_3007.countNodes_3007());
        //menambahkan simpul data 1
        Node_2511533007 root_3007 = new Node_2511533007(1);
        //menjadikan simpul 1 sebagai root
        tree_3007.setRoot_3007(root_3007);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_3007.countNodes_3007());
        Node_2511533007 node2_3007 = new Node_2511533007(2);
        Node_2511533007 node3_3007 = new Node_2511533007(3);
        Node_2511533007 node4_3007 = new Node_2511533007(4);
        Node_2511533007 node5_3007 = new Node_2511533007(5);
        Node_2511533007 node6_3007 = new Node_2511533007(6);
        Node_2511533007 node7_3007 = new Node_2511533007(7);
        Node_2511533007 node8_3007 = new Node_2511533007(8);
        Node_2511533007 node9_3007 = new Node_2511533007(9);
        root_3007.setLeft_3007(node2_3007);
        node2_3007.setLeft_3007(node4_3007);
        node2_3007.setRight_3007(node5_3007);
        node4_3007.setRight_3007(node8_3007);
    }
}

```

```

        root_3007.setRight_3007(node3_3007);
        node3_3007.setLeft_3007(node6_3007);
        node3_3007.setRight_3007(node7_3007);
        node6_3007.setLeft_3007(node9_3007);
        //Set root
        tree_3007.setCurrent_3007(tree_3007.getRoot_3007());
        System.out.println("menampilkan simpul terakhir");
        System.out.println(tree_3007.getCurrent_3007().getData_3007());
        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
        System.out.println(tree_3007.countNodes_3007());
        System.out.println("InOrder: ");
        tree_3007.printInorder_3007();
        System.out.println("\nPreOrder: ");
        tree_3007.printPreOrder_3007();
        System.out.println("\nPostOrder");
        tree_3007.printPostOrder_3007();
        System.out.println("\nDmenampilkan simpul dalam bentuk pohon: ");
        tree_3007.print_3007();
    }
}

```

Kode Program 2.3

Class ini hanya memiliki satu method utama, yaitu main, yang berfungsi sebagai titik awal jalannya program untuk membuat objek tree, menyusun struktur hierarki sembilan simpul biner, dan menguji seluruh fungsi manipulasi serta pencetakan tree tersebut ke layar.

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon:
|
|-- 7
|
|-- 3
|   |-- 6
|   |-- 9
|-- 1
|   |-- 5
|   |-- 2
|       |-- 8
|       |-- 4

```

2.4 Kode Program GraphTraversal

```

package pekan9_2511533007;
import java.util.*;

public class GraphTraversal_2511533007 {
    private Map<String, List<String>> graph_3007 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_3007(String node1_3007, String node2_3007) {
        graph_3007.putIfAbsent(node1_3007, new ArrayList<>());
        graph_3007.putIfAbsent(node2_3007, new ArrayList<>());
        graph_3007.get(node1_3007).add(node2_3007);
        graph_3007.get(node2_3007).add(node1_3007);
    }

    // Menampilkan graf awal
    public void printGraph_3007() {

```



```

        System.out.println("Graf Awal (Adjacency List):");
        for (String node_3007 : graph_3007.keySet()) {
            System.out.print(node_3007 + " -> ");
            List<String> neighbors_3007 = graph_3007.get(node_3007);
            System.out.println(String.join(", ", neighbors_3007));
        }
        System.out.println();
    }

    // DFS rekursif
    public void dfs_3007(String start_3007) {
        Set<String> visited_3007 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_3007(start_3007, visited_3007);
        System.out.println();
    }

    private void dfsHelper_3007(String current_3007, Set<String> visited_3007) {
        if (visited_3007.contains(current_3007)) return;
        visited_3007.add(current_3007);
        System.out.print(current_3007 + " ");
        for (String neighbor_3007 : graph_3007.getDefault(current_3007, new ArrayList<>())) {
            dfsHelper_3007(neighbor_3007, visited_3007);
        }
    }

    //BFS iteratif
    public void bfs_3007(String start_3007) {
        Set<String> visited_3007 = new HashSet<>();
        Queue<String> queue_3007 = new LinkedList<>();
        queue_3007.add(start_3007);
        visited_3007.add(start_3007);
        System.out.println("Penelusuran BFS:");
        while (!queue_3007.isEmpty()) {
            String current_3007 = queue_3007.poll();
            System.out.print(current_3007 + " ");
            for (String neighbor_3007 : graph_3007.getDefault(current_3007, new ArrayList<>())) {
                if (!visited_3007.contains(neighbor_3007)) {
                    queue_3007.add(neighbor_3007);
                    visited_3007.add(neighbor_3007);
                }
            }
        }
        System.out.println();
    }

    // main
    public static void main(String[] args) {
        GraphTraversal_2511533007 graph_3007 = new GraphTraversal_2511533007();

        //contoh graf: A-B, A-C, B-D, B-E
        graph_3007.addEdge_3007("A", "B");
        graph_3007.addEdge_3007("A", "C");
        graph_3007.addEdge_3007("B", "D");
        graph_3007.addEdge_3007("B", "E");
        // cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_3007.printGraph_3007();
        // lakukan penelusuran
        graph_3007.dfs_3007("A");
        graph_3007.bfs_3007("A");
    }
}

```

Kode Program 2.4

Method `addEdge_3007` berfungsi menambahkan hubungan timbal balik antar dua simpul pada graf tak berarah, sedangkan `printGraph_3007` digunakan untuk menampilkan daftar tetangga dari setiap simpul. Method `dfs_3007` bertindak sebagai pemanggil awal yang

menyiapkan media untuk melacak sebelum memicu method privat dfsHelper_3007 untuk menelusuri graf secara mendalam (DFS) menggunakan rekursi. Method bfs_3007 mengimplementasikan penelusuran graf secara melebar (BFS) menggunakan antrean (queue). Terakhir, method main berfungsi sebagai pengendali utama untuk membuat objek graf contoh, membangun relasi antar simpul, menampilkan struktur graf, serta menjalankan pengujian penelusuran DFS dan BFS.

```
Graf Awal adalah:  
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa Binary Tree dan Graph Traversal merupakan struktur data non linear yang sangat efektif untuk merepresentasikan hubungan hierarki. Pada Binary Tree, struktur hierarki dibangun menggunakan representasi simpul yang maksimal memiliki dua cabang anak (kiri dan kanan), di mana struktur ini dapat ditelusuri secara teratur menggunakan tiga metode rekursif, yaitu *Preorder* (parent, kiri, kanan), *Inorder* (kiri, parent, kanan) untuk menghasilkan urutan data yang terstruktur, dan *Postorder* (kiri, kanan, parent). Pada Graph Traversal, relasi antar simpul berhasil diimplementasikan menggunakan pemetaan daftar ketetanggaan (*Adjacency List*). Penelusuran graf dilakukan menggunakan dua algoritma utama dengan karakteristik yang berbeda, yaitu DFS yang memanfaatkan prinsip rekursi untuk menjelajahi simpul sedalam mungkin ke satu arah terlebih dahulu, serta BFS yang memanfaatkan struktur data *Queue* untuk menjelajahi simpul secara melebar level demi level.

DAFTAR PUSTAKA

- [1] Universitas Telkom Jakarta, "Pengenal Tree dan Graph dalam Struktur Data," [Daring]. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/pengenalan-tree-dan-graph-dalam-struktur-data/>. [Diakses: 11-Juni-2026].
- [2] W3Schools, "DSA Graphs Traversal," [Daring]. Tersedia pada: https://www.w3schools.com/dsa/dsa_algo_graphs_traversal.php. [Diakses: 11-Juni-2026].

Laporan Tugas

A. Kode Program

```
package pekan9_2511533007;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import java.util.Stack;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.SwingUtilities;

public class PetaStasiun_2511533007 extends JFrame {

    private static final long serialVersionUID = 1L;

    // Inisialisasi Komponen GUI
    private JComboBox<String> cbAwal_3007;
    private JComboBox<String> cbTujuan_3007;
    private JButton btnBfs_3007, btnDfs_3007, btnReset_3007;
    private JTextArea txtGraph_3007;
    private JTextArea txtHasil_3007;

    // Inisialisasi struktur data
    private Map<String, List<String>> graph_3007 = new LinkedHashMap<>();

    public PetaStasiun_2511533007() {
        // Setup Frame
        setTitle("Pencarian Jalur BFS & DFS Stasiun Kereta Api");
        setSize(850, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel inputPanel_3007 = new JPanel(new FlowLayout());
        inputPanel_3007.add(new JLabel("Lokasi Awal:"));

        String[] stasiun_3007 = {"padang", "padang pariaman", "bukittinggi", "payakumbuh", "pasa baru", "ulak karang", "tunggul hitam", "pauh", "limau manih", "unand"};
        cbAwal_3007 = new JComboBox<>(stasiun_3007);
        inputPanel_3007.add(cbAwal_3007);

        inputPanel_3007.add(new JLabel("Lokasi Tujuan:"));
        cbTujuan_3007 = new JComboBox<>(stasiun_3007);
        inputPanel_3007.add(cbTujuan_3007);

        JPanel panelGraph_3007 = new JPanel(new BorderLayout());
        panelGraph_3007.setBorder(BorderFactory.createTitledBorder("Visualisasi Graph Stasiun"));
        txtGraph_3007 = new JTextArea();
        txtGraph_3007.setEditable(false);
        txtGraph_3007.setFont(new Font("Monospaced", Font.PLAIN, 12));
        txtGraph_3007.setBackground(new Color(245, 245, 245));
        JScrollPane scrollGraph_3007 = new JScrollPane(txtGraph_3007);
```



```

        "      |                               /      |                               |\n" +
        "      | pasa baru -----      | pauh -----      limau manih\n" +
        "      |                               |      \\\n" +
        "      |                               |      \\\n" +
        "      | unand ---- padang pariaman ---- bukittinggi ----- payakumbuh\n"
    );
}

// Melakukan pencarian menggunakan BFS
private void BFS_3007() {
    search_3007(true);
}

// Melakukan pencarian menggunakan DFS
private void DFS_3007() {
    search_3007(false);
}

// Algoritma Pencarian
private void search_3007(boolean isBfs_3007) {
    String start_3007 = (String) cbAwal_3007.getSelectedItem();
    String goal_3007 = (String) cbTujuan_3007.getSelectedItem();

    List<String> visitedOrder_3007 = new ArrayList<>();
    Map<String, String> parent_3007 = new HashMap<>();
    boolean found_3007 = false;

    if (isBfs_3007) {
        Queue<String> queue_3007 = new LinkedList<>();
        Set<String> visited_3007 = new HashSet<>();

        queue_3007.add(start_3007);
        visited_3007.add(start_3007);

        while (!queue_3007.isEmpty()) {
            String curr_3007 = queue_3007.poll();
            visitedOrder_3007.add(curr_3007);

            if (curr_3007.equals(goal_3007)) {
                found_3007 = true;
                break;
            }

            for (String neighbor_3007 : graph_3007.get(curr_3007)) {
                if (!visited_3007.contains(neighbor_3007)) {
                    visited_3007.add(neighbor_3007);
                    parent_3007.put(neighbor_3007, curr_3007);
                    queue_3007.add(neighbor_3007);
                }
            }
        }
    } else {
        Stack<String> stack_3007 = new Stack<>();
        Set<String> visited_3007 = new HashSet<>();

        stack_3007.push(start_3007);

        while (!stack_3007.isEmpty()) {
            String curr_3007 = stack_3007.pop();

            if (!visited_3007.contains(curr_3007)) {
                visited_3007.add(curr_3007);
                visitedOrder_3007.add(curr_3007);

                if (curr_3007.equals(goal_3007)) {
                    found_3007 = true;
                    break;
                }
            }

            // Reverse agar DFS konsisten dengan urutan input visual kiri-kanan
            List<String> neighbors_3007 = new ArrayList<>(graph_3007.get(curr_3007));
            Collections.reverse(neighbors_3007);

            for (String neighbor_3007 : neighbors_3007) {

```

```

        if (!visited_3007.contains(neighbor_3007)) {
            stack_3007.push(neighbor_3007);
            if (!parent_3007.containsKey(neighbor_3007)) {
                parent_3007.put(neighbor_3007, curr_3007);
            }
        }
    }
}

displayPath_3007(start_3007, goal_3007, parent_3007, visitedOrder_3007, found_3007, isBfs_3007
? "BFS" : "DFS");
}

// Menampilkan Jalur yang Ditemukan
private void displayPath_3007(String start_3007, String goal_3007, Map<String, String>
parent_3007,
                                List<String> visitedOrder_3007, boolean found_3007, String
method_3007) {
    StringBuilder pathLog_3007 = new StringBuilder();

    if (found_3007) {
        List<String> route_3007 = new ArrayList<>();
        String step_3007 = goal_3007;
        while (step_3007 != null) {
            route_3007.add(step_3007);
            step_3007 = parent_3007.get(step_3007);
        }
        Collections.reverse(route_3007);
        pathLog_3007.append(String.join(" -> ", route_3007));
    } else {
        pathLog_3007.append("Jalur tidak ditemukan!");
    }

    txtHasil_3007.setText("");
    txtHasil_3007.append("Metode Pencarian      : " + method_3007 + "\n");
    txtHasil_3007.append("Lokasi Awal        : " + start_3007 + "\n");
    txtHasil_3007.append("Lokasi Tujuan       : " + goal_3007 + "\n");
    txtHasil_3007.append("-----\n");
    txtHasil_3007.append("Jalur (Path)        : \n" + pathLog_3007.toString() + "\n\n");
    txtHasil_3007.append("Urutan Node Dikunjungi : \n" + String.join(", ", visitedOrder_3007) +
"\n\n");
    txtHasil_3007.append("Jumlah Node Eksplorasi : " + visitedOrder_3007.size() + "\n");
}

// Mengembalikan Kondisi Form ke Awal
private void resetGraph_3007() {
    cbAwal_3007.setSelectedIndex(0);
    cbTujuan_3007.setSelectedIndex(0);
    txtHasil_3007.setText("");
}

// Eksekusi GUI
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        PetaStasiun_2511533007 gui_3007 = new PetaStasiun_2511533007();
        gui_3007.setVisible(true);
    });
}
}

```

B. Penjelasan Kode

PetaStasiun_2511533007() (Constructor) Method ini berfungsi untuk menginisialisasi dan menyusun seluruh komponen antarmuka pengguna (GUI) pada *frame* utama, seperti membuat pilihan menu stasiun, area visualisasi peta, dan kolom

log hasil menggunakan BorderLayout. Selain menata tampilan, method ini juga bertugas memakai *event listener* pada tombol BFS, DFS, dan Reset agar program dapat merespons klik dari pengguna.

initGraph_3007() Method ini bertugas untuk membangun struktur data graf yang merepresentasikan peta stasiun ke dalam program. Prosesnya dilakukan dengan memasukkan sepuluh nama stasiun sebagai titik (*node*) ke dalam LinkedHashMap, kemudian dilanjutkan dengan memanggil fungsi pembuat jalur untuk menghubungkan stasiun stasiun tersebut sebanyak 15 jalur.

addEdge_3007(String a_3007, String b_3007) Method ini berfungsi untuk menciptakan jalur hubungan (*edge*) yang menghubungkan dua stasiun secara di dalam graf. Karena graf ini bersifat tidak berarah (*undirected*), method ini akan mendaftarkan stasiun B ke dalam daftar tetangga stasiun A sekaligus mendaftarkan stasiun A ke dalam daftar tetangga stasiun B agar jalur kereta tersebut dapat dilalui dari kedua arah.

displayGraph_3007() Method ini digunakan khusus untuk menampilkan visualisasi peta stasiun dalam bentuk teks gambar ke dalam komponen txtGraph_3007.

BFS_3007() Method ini merupakan fungsi perantara yang bertugas untuk memicu proses pencarian rute stasiun dengan metode *Breadth First Search*. Ketika tombol BFS diklik, method ini akan langsung memanggil fungsi pencarian utama (search_3007) dengan mengirimkan parameter nilai true sebagai penanda bahwa algoritma yang digunakan adalah pencarian secara melebar.

DFS_3007() Method ini berfungsi untuk memulai proses pencarian rute stasiun dengan metode *Depth First Search*. Ketika tombol DFS diklik, method ini akan langsung memanggil fungsi pencarian utama (search_3007) dengan mengirimkan parameter nilai false sebagai penanda bahwa algoritma yang dijalankan adalah pencarian secara mendalam.

search_3007(boolean isBfs_3007) Method ini merupakan inti dari program yang mengeksekusi algoritma pencarian dari lokasi awal ke stasiun tujuan. Jika parameter bernilai true (BFS), program memanfaatkan struktur data Queue untuk memeriksa stasiun tetangga terdekat terlebih dahulu, sedangkan jika false (DFS),

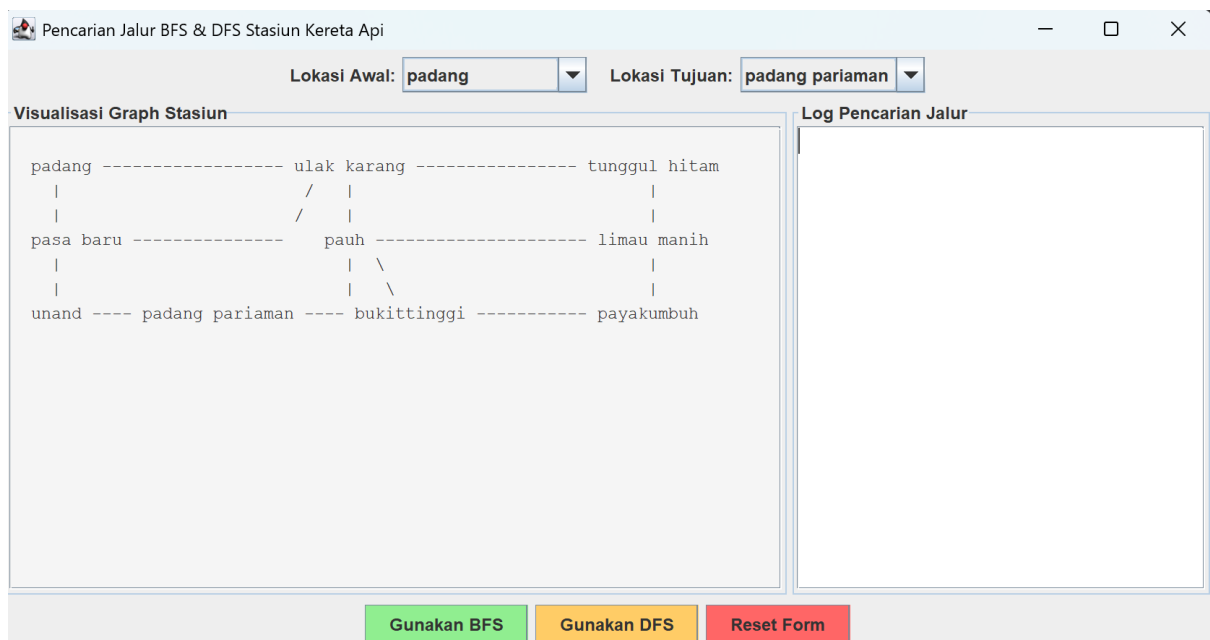
program menggunakan Stack untuk menjelajah jalur sedalam mungkin, sambil merekam urutan kunjungan dan menyimpan relasi parent untuk pembentukan rute.

`displayPath_3007()` Method ini berfungsi untuk mengolah, menyusun, dan menampilkan data hasil pencarian algoritma ke dalam area log teks secara rapi. Fungsi ini akan melacak balik struktur *parent* untuk merekonstruksi rute perjalanan dari awal sampai tujuan, menghitung total stasiun yang sempat diperiksa, lalu mencetaknya ke layar agar bisa dibaca oleh *user*.

`resetGraph_3007()` Method ini berfungsi untuk membersihkan inputan formulir dan mengembalikan kondisi aplikasi ke keadaan semula. Saat dijalankan, method ini akan mengatur ulang pilihan pada kotak menu (*JComboBox*) lokasi awal dan tujuan kembali ke pilihan pertama (indeks 0) serta menghapus seluruh teks log hasil pencarian yang ada pada layar.

`Main()` Method ini merupakan program utama dari seluruh program Java untuk memulai jalannya GUI.

C. Output Kode



Pencarian Jalur BFS & DFS Stasiun Kereta Api

Lokasi Awal: Lokasi Tujuan:

Visualisasi Graph Stasiun

```

padang ----- ulak karang ----- tunggul hitam
|               / |               |
|               / |               |
pasa baru ----- pauh ----- limau manih
|               | \               |
|               | \               |
unand ---- padang pariaman ---- bukittinggi ----- payakumbuh

```

Log Pencarian Jalur

Metode Pencarian : BFS

Lokasi Awal : padang

Lokasi Tujuan : padang pariama

Jalur (Path) :

padang -> ulak karang -> pauh -> pa

Urutan Node Dikunjungi :

padang, ulak karang, pasa baru, tun

Jumlah Node Eksplorasi : 8

Pencarian Jalur BFS & DFS Stasiun Kereta Api

Lokasi Awal: Lokasi Tujuan:

Visualisasi Graph Stasiun

```

padang ----- ulak karang ----- tunggul hitam
|               / |               |
|               / |               |
pasa baru ----- pauh ----- limau manih
|               | \               |
|               | \               |
unand ---- padang pariaman ---- bukittinggi ----- payakumbuh

```

Log Pencarian Jalur

Metode Pencarian : DFS

Lokasi Awal : padang

Lokasi Tujuan : padang pariama

Jalur (Path) :

padang -> ulak karang -> pauh -> pa

Urutan Node Dikunjungi :

padang, ulak karang, tunggul hitam,

Jumlah Node Eksplorasi : 8